

Networking Fundamentals - System Design Part 2

▼ 1. OSI Model

Modern distributed systems rely on multiple layers of communication, from physical network transmission to high-level application protocols.

The **OSI (Open Systems Interconnection) Model** provides a conceptual framework that divides network communication into seven distinct layers. Each layer has a specific responsibility, from transmitting raw bits over a network to delivering structured data to applications.

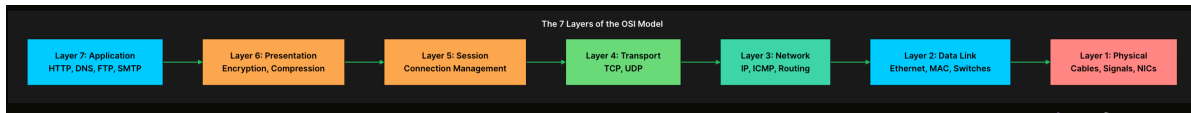
In practice, most real-world systems use simplified models like **TCP/IP**, but the OSI model remains one of the best ways to understand how data moves through a network stack and where different technologies operate.

Why Does the OSI Model Exist?

In the early days of networking, every vendor did things differently. IBM had its own networking architecture. So did Honeywell. So did DEC. If you bought hardware from one vendor, you were stuck with them. Their devices couldn't talk to anyone else's.

In 1984, the International Organization for Standardization (ISO) published the OSI model as a universal reference framework. The idea was straightforward: if everyone agrees on what each layer should do, different vendors can build products that actually work together.

The OSI model doesn't specify *how* to implement networking. It specifies *what responsibilities* each layer has. Think of it as a contract. Layer 3 promises to handle routing. Layer 4 promises to handle reliable delivery. As long as each layer keeps its promise, the whole system works, no matter who built the hardware or software.



Mnemonic: Please Do Not Throw Sausage Pizza Away (Physical, Data Link, Network, Transport, Session, Presentation, Application).

▼ Layer 1: Physical

The Physical layer deals with raw bits traveling over a medium. Electrical signals on copper wire. Light pulses through fiber optic cables. Radio waves for WiFi. This layer just moves them from point A to point B. It is responsible for keeping sender and receiver in sync at the bit level.

The devices here are hubs, repeaters, cables, and Network Interface Cards (NICs).

▼ Layer 2: Data Link

The Physical layer can move bits, but it has no concept of structure or addressing. The Data Link layer fixes that by organizing bits into **frames** and introducing **MAC addresses**, unique hardware identifiers burned into every network interface.

MAC address is 48 bits long and uniquely identifies a device on a local network.

Responsibilities of Data Link Layer

1. Framing: package bits into frames.
2. Use mac addresses to identify devices on a local network
3. detect transmission errors using checksums (frame check seq at the end of each frame)
4. Control which device gets to use the shared medium at any given time.

Key device at this layer is Switch. It reads the destination mac address in each frame, looks it up in mac address table and forwards the frame only to the correct port. Compared that to a hub (Layer 1), which blindly broadcasts every signal to every port.

Layer 2 only works within a single network. To reach a device on a different network, you need Layer 3.

▼ Layer 3: Network

The Network layer is what lets your laptop in New York talk to a server in Tokyo. It **handles routing**: figuring out a path across multiple networks to reach the destination.

Responsibilities:

1. Assigns logical addresses (IP addresses)
2. Routes packets between networks
3. breaks up packets that are too large for a link (fragmentation)
4. figures out the best path to the destination.

key device at this layer is Router. They read the destination IP address in each packet and forward it toward its destination, one hop at a time.

Layer 3 only gets packets to the right machine. It doesn't know which application on that machine should receive the data. That's where the Transport layer comes in

▼ MAC addresses vs IP addresses

MAC addresses are permanent, burned into hardware. IP addresses are logical and can change. You need both: MAC addresses handle local delivery within a network (Layer 2), while IP addresses handle routing across networks (Layer 3).

▼ Layer 4: Transport

This layer provides **ports**, numbers that identify which application should handle incoming data. For ex: A web server listens on port 443 (HTTPS). An SSH server listens on port 22. The combination of IP address + port is what we call a socket.

TCP and UDP are the two protocols we see here.

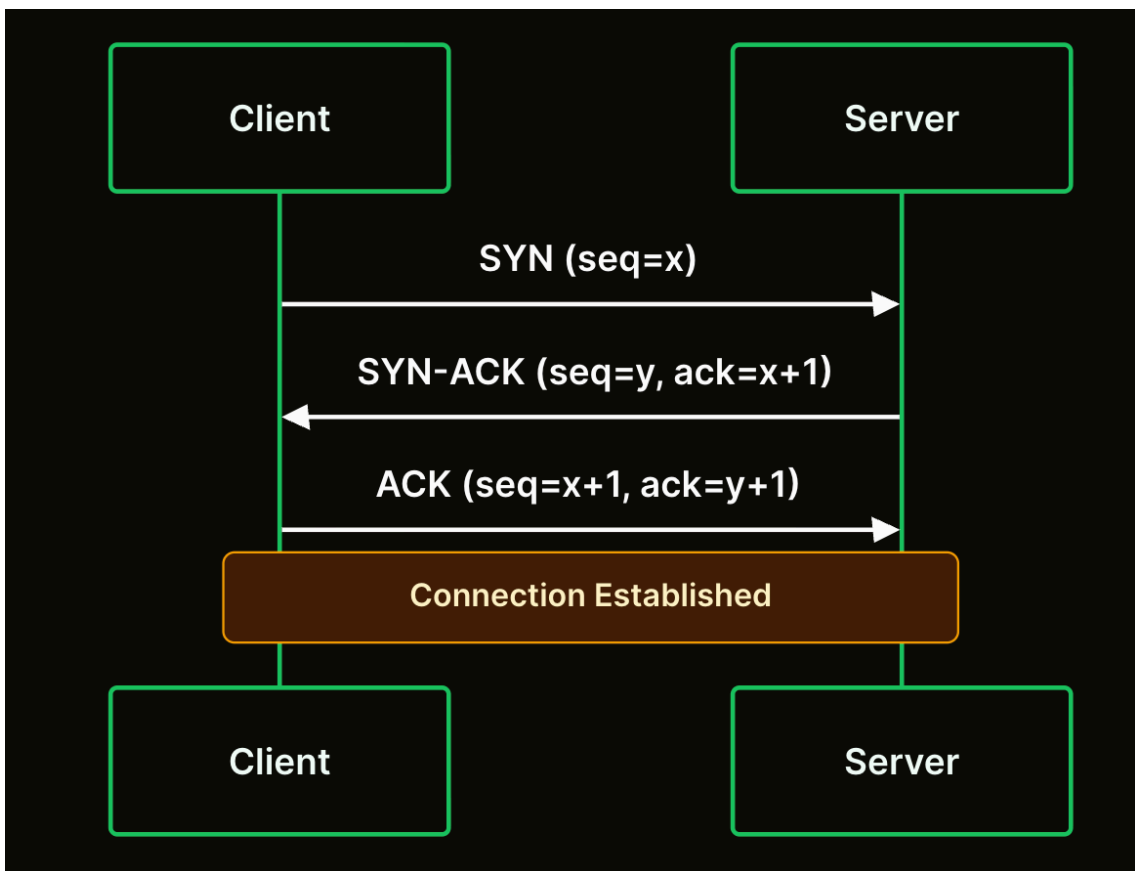
Responsibilities:

1. Breaking large chunks of data into smaller segments.

2. Flow Control: managing flow control so a fast sender doesn't overwhelm a slow receiver.

Feature	TCP	UDP
Connection	3-way handshake required	Connectionless
Reliability	Guaranteed delivery, retransmission	Best effort, no guarantees
Ordering	Packets arrive in order	Order not guaranteed
Overhead	Higher (headers, acknowledgments)	Minimal
Use Cases	HTTP, email, file transfer	DNS lookups, video streaming, gaming

Before any data flows, TCP establishes a connection using a three-way handshake:



▼ Layer 5: Session

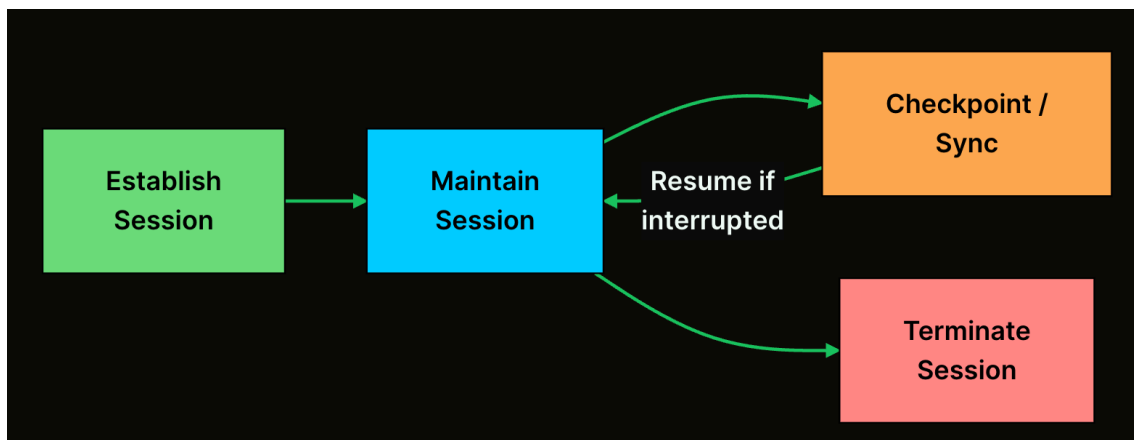
This layer manages the lifecycle of connections between applications: setting them up, keeping them alive, and tearing them down.

In practice, most modern protocols fold this into the Application layer.

Use case:

When you download a large file and the connection drops, session management decides whether you restart from scratch or resume from where you left off.

When you're on a video call and your WiFi hiccups for a second, the session layer is what keeps things going.



▼ Layer 6: Presentation

This layer acts as the translator. It converts data between the format applications use and the format the network needs.

Responsibilities: 3 things happen here

1. Encryption (TLS/SSL encrypts data before it hits the wire).
2. Compression (gzip or Brotli shrink the data to save bandwidth).
3. Encoding (converting between character sets like ASCII and UTF-8, or data formats like JPEG and JSON).

Like Session layer, Presentation layer is also folded in Application Layer in real implementations

Function	Examples
Encryption	TLS/SSL, AES
Compression	gzip, Brotli, deflate
Encoding	ASCII, UTF-8, Base64
Data Formats	JPEG, PNG, MPEG, JSON

▼ Layer 7: Application

This is the layer we interact with most directly. Everytime our browsers sends at HTTP request, every time we send an email, everytime our code makes a DNS query, that's layer 7 at work.

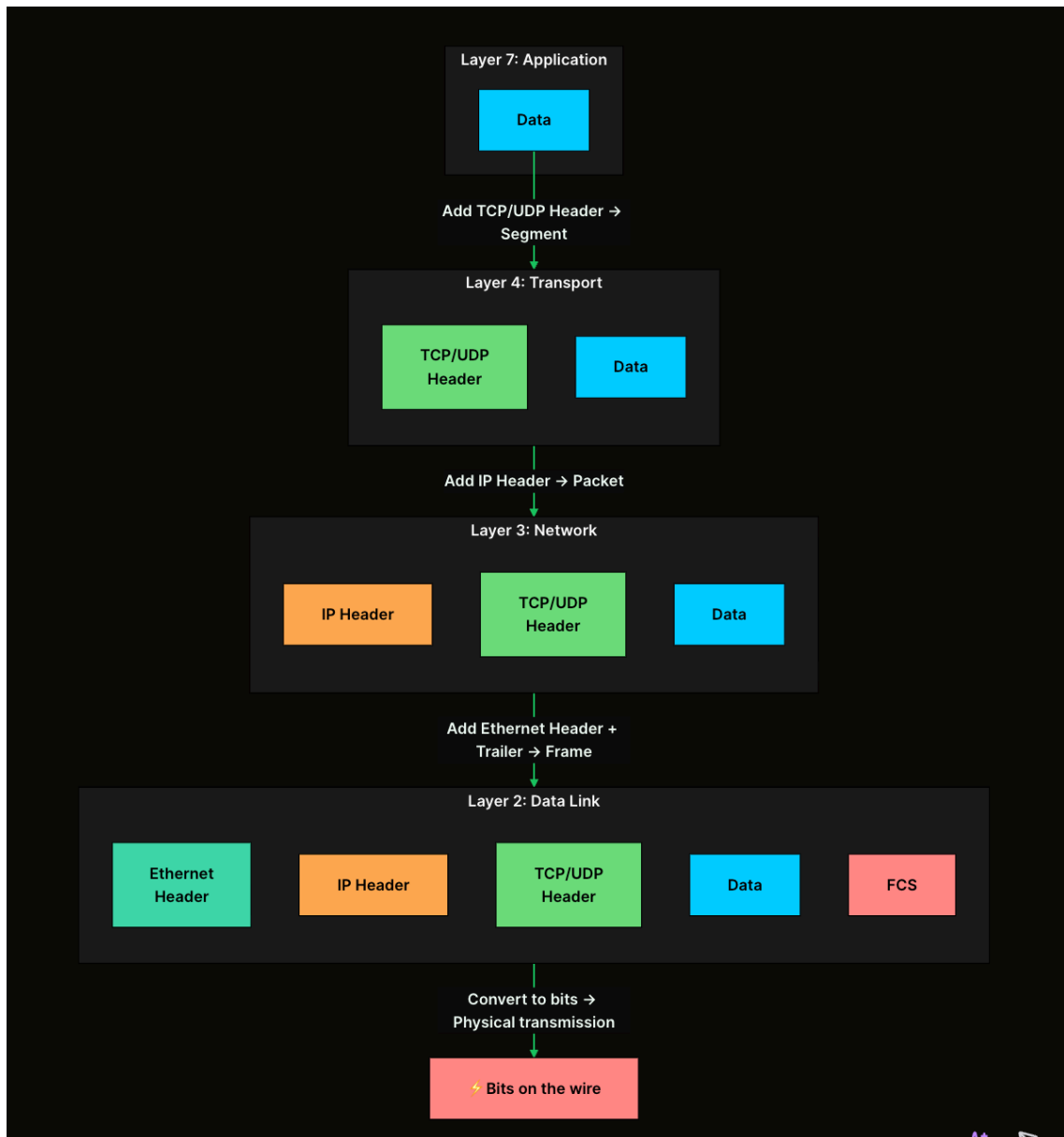
It's where all the high-level protocols live: HTTP(443), FTP, SMTP(25), DNS, SSH(22).

▼ How data flows?

When we send data over a network, it doesn't teleport from our application to the wire. It passes down through each layer and each layer wraps the data with its own header. This wrapping process is called **encapsulation**.

1. The **Application layer** generates the data, say an HTTP request
2. The **Transport layer** wraps it with a TCP or UDP header (source port, destination port, sequence numbers), creating a **segment**
3. The **Network layer** wraps that with an IP header (source IP, destination IP), creating a **packet**
4. The **Data Link layer** wraps that with an Ethernet header and trailer (source MAC, destination MAC, checksum), creating a **frame**
5. The **Physical layer** converts the frame into raw **bits** and sends them over the wire.

On the receiving end, the reverse happens. This is **decapsulation**. Each layer strips off its header, reads what it needs, and passes the payload up. By the time data reaches Layer 7, all the networking headers are gone and the application gets clean data.

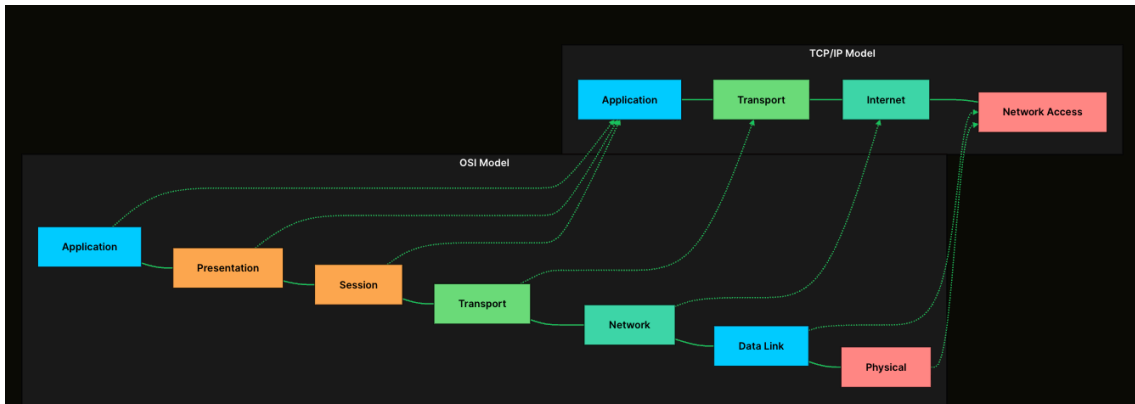


Each layer gives its own name to the data unit it works with:

Layer	Data Unit Name
Application (Layer 7)	Data
Transport (Layer 4)	Segment (TCP) / Datagram (UDP)
Network (Layer 3)	Packet
Data Link (Layer 2)	Frame
Physical (Layer 1)	Bits

▼ OSI vs TCP/IP Model

While OSI model is a powerful theoretical tool, the **TCP/IP model** is the one that's actually implemented and used by the Internet. It's a more pragmatic, consolidated model, and has only four layers.



Why bother learning the OSI model at all?

OSI model offers better mental model for reasoning about problems. With 7 layers, you can pinpoint where an issue lives. For instance, when someone says this is a Layer 4 issue, everyone knows its related to transport layer, ports, TCP/UDP.

The TCP/IP model's "Application" layer lumps too many things together to be useful for that kind of precision.

▼ 2. IP Address

Every device connected to the internet needs a unique way to be identified. Without a clear addressing system, data would have no reliable way to reach its destination. This is where **IP (Internet Protocol) addresses** come in.

An **IP address** is a unique numerical identifier assigned to a device on a network. It allows routers and networking infrastructure to determine where data should be sent and how it should be routed across the internet.

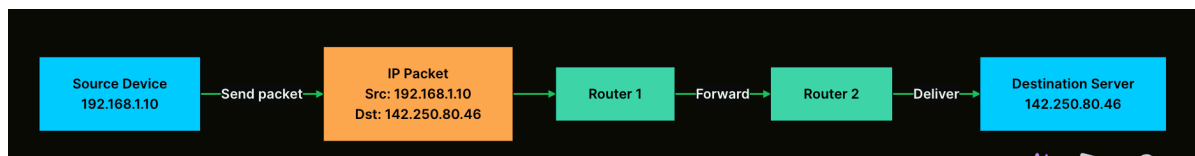
Without IP addresses, there's no way for your laptop to tell a router "send this packet to that web server." Every device that sends or receives data needs one.

There are 2 versions:

IPv4: 32 bits addresses, 2^{32} possible addresses which is roughly around 4.3 billion

IPv6: 128 bits addresses

IP addresses operate at **Layer 3 (Network layer)** of the OSI model. This is the layer responsible for routing, for getting packets from one network to another across the internet. Layer 2 (Data Link) handles local delivery using MAC addresses, but once a packet needs to leave your local network, IP takes over.



Every IP packet carries two addresses: the **source IP** (where it came from) and the **destination IP** (where it's going). Routers read the destination IP and forward the packet one hop closer to its target.

▼ Address Classes / Classful addressing (Historical approach)

In the early internet, addresses were divided into **classes** based on the first few bits. This determined how the 32 bits were split between the network portion (which network?) and the host portion (which device on that network?).

This rigid system burned thru the available IPv4 address space fast.

Later on this was replaced by subnetting and CIDR.

▼ Subnetting and CIDR Notation

Under classful addressing, you could only split addresses along fixed boundaries: /8, /16, or /24. If you needed 1,000 addresses, you had to allocate a /16 block with 65,536 addresses. That's a 98.5% waste. The internet was growing rapidly and this waste was unsustainable. To solve this, the concept of subnet masks and CIDR(classless inter domain routing) notation was introduced.

Every IP address is divided into two parts: a **network portion** (which network is this device on?) and a **host portion** (which device on that

network?). The **subnet mask** determines where this split happens.

In CIDR notation, the split is written as a suffix. `192.168.1.0/24` means the first 24 bits are the network portion and the remaining 8 bits are for hosts. That gives you $2^8 = 256$ addresses (254 usable, since the first is the network address and the last is the broadcast address).

What makes CIDR flexible is that the split can happen at any bit position, not just on octet boundaries. Need 1,000 addresses? Use a `/22` block (1,024 addresses). Need 16? Use a `/28`. No more waste.

CIDR calculation:

Let's work through `10.0.0.0/20`:

- `/20` means 20 bits for the network, 12 bits for hosts
- Host addresses = $2^{12} = 4,096$ total (4,094 usable)
- Range: `10.0.0.0` to `10.0.15.255`

Cloud platforms use the concept of CIDR blocks. When you create a VPC, you define its IP range using a CIDR. For instance, a VPC with `10.0.0.0/16` gives you 65,536 addresses. You then carve it into subnets: `10.0.1.0/24` for web servers, `10.0.2.0/24` for databases, `10.0.3.0/24` for internal services.

Choosing the right CIDR block size upfront matters. Too small, and you run out of addresses. Too large, and you waste address space that could conflict with other VPCs if you ever need to peer them.

NOTE: Not all of these addresses work the same way. Some can reach the internet, and some can't.

▼ Public vs Private IP addresses

The difference comes down to who can see the address. Some IPs are routable on the public internet. Others only work within a private network.

A **public IP address** is globally unique and reachable from anywhere on the internet.

Public IP's are a scarce resource.

Private IPs are not routable on the public internet, which means a router on the internet will simply drop a packet destined for a private IP.

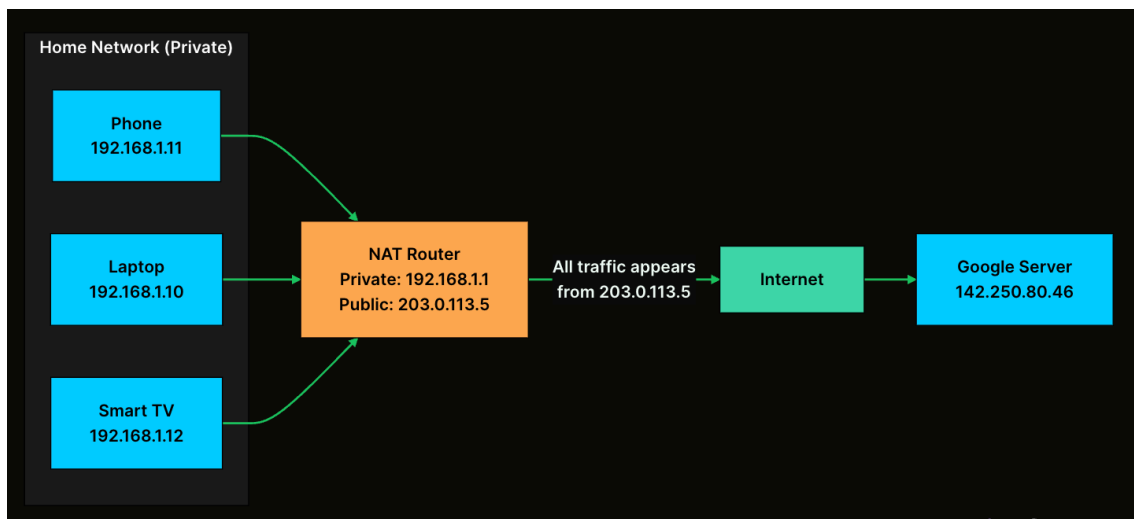
Your laptop right now probably has an IP like `192.168.1.x`. Your neighbor's laptop might also be `192.168.1.x`. That's fine because **private addresses only need to be unique within their own network**.

▼ NAT's and IPv6

NAT (network address translation):

if private addresses are not reachable, then how does your laptop browse through the web. Through NAT.

Your home router has one public IP assigned by your ISP. When your laptop (say, `192.168.1.10`) sends a request to `142.250.80.46` (Google), the router rewrites the source IP from `192.168.1.10` to its own public IP (say, `203.0.113.5`). It keeps a translation table so it knows which internal device to forward the response back to.



NAT is what allowed the internet to keep growing long after IPv4 addresses should have run out. Billions of devices share a much smaller pool of public IPs.

In cloud setups, the same split of public / private IPs applies. Your EC2 instances, kubernetes pods, RDS databases they all get private IPs within

a VPC. Only load balancers and NAT gateways get public IPs that face the internet. This is both a cost optimization(public IPs are limited and hence sometimes charged) and a security measure (internal services becomes unreachable from the outside world).

NAT was a workaround for IPv4 address exhaustion problem. It's true fix came with IPv6.

IPv4's supports around 4.3 billion devices. That number seemed massive in 1990s, but now there are over 15 billion devices and that number keeps growing with IOTs, phones, Laptops, security cameras, cars, industrial sensors, they all need IP addresses. NATs has stretched IPv4s lifespan but it adds complexity, breaks certain protocols and makes end to end connectivity harder to manage. IPv6 was designed to solve this permanently. With 128 bits, it supports 2^{128} addresses which is enough to assign a unique IP to every atom on Earth's surface and still have addresses left over.

Some IP addresses have special meaning:

1. **Loopback (127.0.0.1)**: The entire 127.0.0.0/8 range is reserved for loopback, but 127.0.0.1 is what everyone uses. When you run a web server locally and visit localhost:3000 , your traffic never leaves your machine. It goes down the network stack, hits the loopback interface, and comes right back up. Useful for development and testing, and also for inter-process communication on the same host.
2. **Link-local (169.254.x.x)**: If your computer can't reach a DHCP server, it assigns itself an address in this range. Seeing 169.254.x.x on a device almost always means DHCP is broken or the network is misconfigured.
3. **0.0.0.0** : This means different things depending on context. In a server's bind address, 0.0.0.0:8080 means "listen on port 8080 on all network interfaces." In a routing table, 0.0.0.0/0 is the **default route**, the catch-all that matches any destination address.

Knowing what these addresses mean helps you debug faster. Seeing 169.254.x.x immediately tells you DHCP is the problem.

Seeing `0.0.0.0` in a bind configuration tells you the service is listening everywhere.

Once a packet has a destination IP, how does it get there?

▼ How IP routing works?

When your computer sends a packet to `142.250.80.46`, that packet doesn't take a direct path. It hops through a series of routers, each one making a forwarding decision based on its **routing table**.

Every router maintains a routing table: a list of network prefixes and which next hop to forward matching packets to. When a packet arrives, the router compares the destination IP against all entries in the table and picks the most specific match. This is called **longest prefix match**.

No single router knows the complete path to the destination. Each router only knows the next hop. The packet gets forwarded one router at a time until it arrives.

A typical internet request passes through **10 to 15 routers**. You can see the path yourself using `traceroute`:

```
$ traceroute google.com
1  192.168.1.1      1.2 ms    (your home router)
2  10.0.0.1         5.8 ms    (ISP gateway)
3  72.14.215.85     8.3 ms    (ISP backbone)
4  108.170.252.1    9.1 ms    (Google edge)
5  142.250.80.46    9.5 ms    (destination)
```

TTL : preventing infinite loops

What if routers have a misconfiguration and a packet ends up going in circles? The **TTL (Time to Live)** field prevents this. Every IP packet starts with a TTL value (usually 64 or 128). Each router decrements it by 1. When TTL hits 0, the router drops the packet and sends an ICMP "Time Exceeded" message back to the sender.

▼ BGP : glue of the internet

At the large scale, the internet is a collection of **Autonomous Systems (AS)**, each operated by a different organization (ISPs, cloud providers, large enterprises).

BGP (Border Gateway Protocol) is how these systems exchange routing information and decide how to reach each other.

BGP is what makes the internet work as a whole. It's also what breaks the internet when things go wrong.

▼ 3. DNS

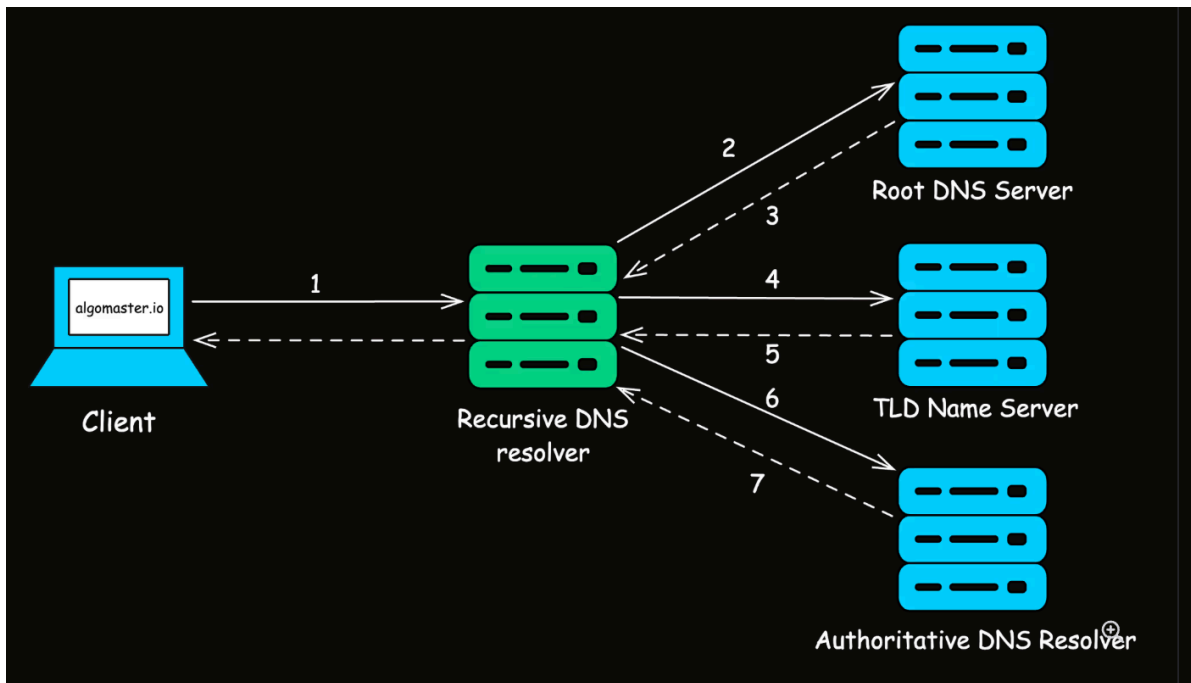
You cannot remember the IP address of every website that you want to visit. This is where DNS (domain name service) comes in.

The **Domain Name System (DNS)** is a distributed, hierarchical system designed to translate human-readable domain names into IP addresses, which are necessary for locating and communicating with the servers on the internet.

So instead of typing the actual IP in the browser, the user just specifies the domain name and DNS takes care of finding the corresponding IP address.

A series of steps occur to translate that domain name into IP.

1. The user / client types the domain name in the browser.
2. DNS Resolver (Recursive Resolver): Our computer sends a request to a DNS resolver.
3. The resolver first queries the root name server (which doesn't know the exact IP) but can direct the resolver to the appropriate TLD server.
 - a. TLD — top level domain like .com, .net, .org etc
4. The resolver then contacts the TLD server for further guidance.
5. Finally the resolver queries the Authoritative Name Server to obtain the precise IP address.
6. The IP address is returned to your browser and you're directed to the website.



NOTE: The resolver takes full responsibility for retrieving the requested IP, making multiple queries on behalf of the client until it gets an answer. The resolver may receive partial answers and is directed to the next server in the hierarchy, with each server providing the best answer it can.

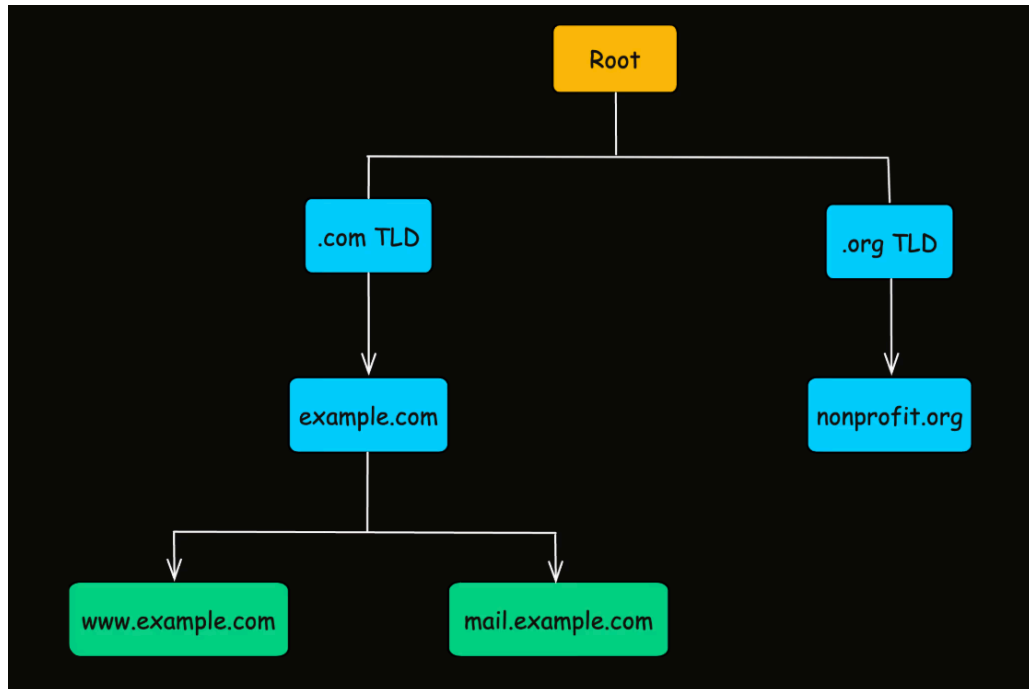
DNS Hierarchy

DNS is designed as a hierarchical system, much like a tree. For ex: at the top are the **root servers**, which know where to find the TLD servers

Then in the next level we have **TLD servers** like .com, .net, .org etc

Then we have **second level domains** — these are domains which we register like for example in example.com

and finally **subdomains** — these are added to further organize the domain



This hierarchy allows DNS to scale efficiently and handle a massive number of queries simultaneously.

Caching in DNS

Caching is a crucial aspect of DNS that stores the output of previously resolved queries for a predetermined period of time(TTL). It helps in improving the performance. Now because of caching, if another user requests the IP for the same domain again, the resolver will directly return the cached response without going thru the entire resolution process again.

Advantages:

1. Reduces Latency: faster responses for frequently accessed domains
2. Lower load on servers: fewer queries are sent upstream, reducing the load on root and TLD servers
3. Improves scalability: caching is what allows DNS to handle massive volume of queries simultaneously.

▼ 4. Proxy and Reverse Proxy

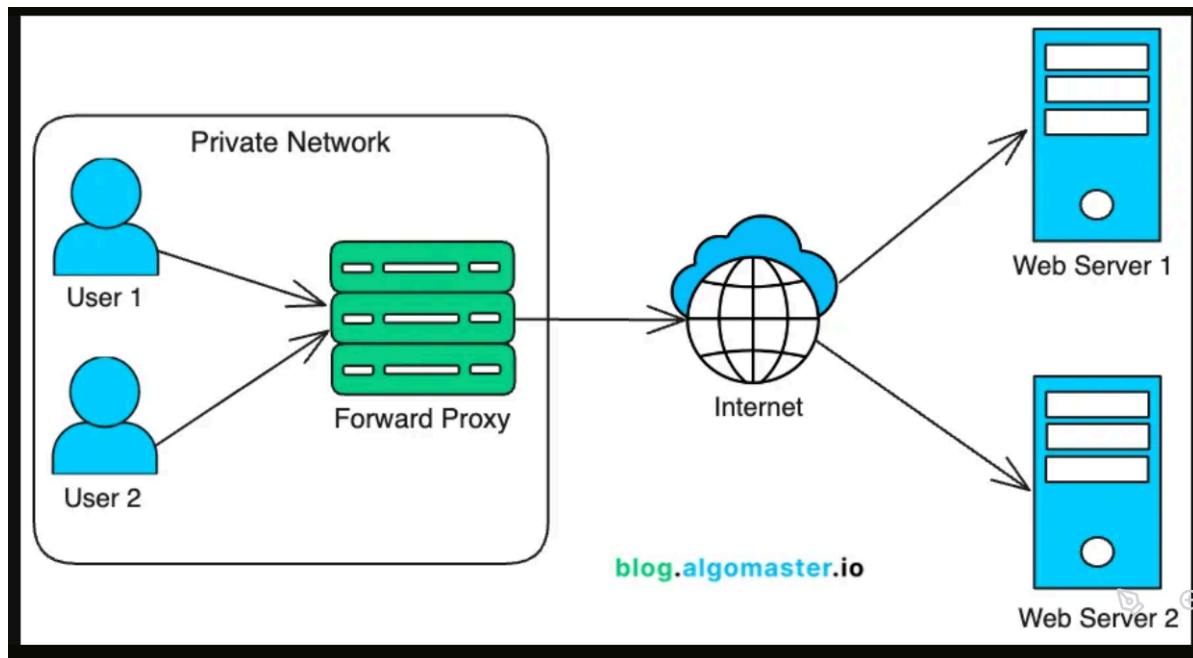
First of all what are proxies? Proxies are entities that has the authority to act on behalf of others.

In computer terms, Proxies and Reverse Proxies are servers that sit between clients and servers to improve security, privacy and performance.

A **Proxy server** (sometimes called a **Forward proxy**) acts on behalf of clients, while a **Reverse Proxy** acts on **behalf** of servers.

So when you send a request, like opening a webpage, the proxy intercepts it, forwards it to the target server, and then relays the server's response back to you.

You can think of it as a middleman, that sits between a private network and public internet.



1. The user types a website URL into their browser. The request is intercepted by the proxy server instead of going directly to the website.
2. The proxy server examines the request to decide if it should forward it, deny it, or serve a cached copy.
3. If the proxy decides to forward the request, it contacts the target server. The backend server's only see the Proxies IP and not the user's IP. (users are hidden)

4. When the target server responds, proxies receives the response and relays it back to the client.

Key Benefits of having a Forward Proxy:

1. **Privacy & Anonymity:** Proxy servers hide your IP address by using their own, so the destination server cannot know your real location or identity.
2. **Access Control:** Organizations use proxies to enforce content restrictions, monitor internet usage.
3. **Enhanced Security:** Proxies can filter out malicious content and block suspicious sites, providing an additional layer of security.
4. **Improved Performance:** Proxies cache frequently accessed content, reducing latency and improving load times for websites.

Real world usecases of Forward Proxy

1. Bypassing geographical restrictions: One of the most common uses of proxy servers is **bypassing geographic restrictions** on websites and content. Streaming services, for instance, often offer different content based on a user's location. With a proxy server based in the target region, you can access that region's content library as if you were a local user.
 - a. **Example:** Suppose you're in India and want to access the US library of a streaming platform (eg.. Netflix). By connecting to a proxy server located in the US, your request to the streaming platform will appear to be coming from the US, allowing access to its content as if you were a US-based viewer
2. **Speed and Performance Optimization (Caching):**
 - a. Proxies can store cached versions of frequently accessed content, enabling faster load times and reducing bandwidth usage.
 - b. When a user requests cached content, the proxy server serves the stored copy rather than fetching it from the destination server, which reduces latency.
 - c. To avoid stale content, it uses a Time-To-Live (TTL) value, automatically expiring cached data after the configured time

- d. **Example:** An organization with hundreds of employees frequently accessing the same online resources can deploy a caching proxy. This proxy caches common websites in its database, so subsequent requests are served quickly from the proxy's storage, saving time and bandwidth.

Reverse Proxy

- A **reverse proxy** is the **reverse** of a forward proxy. It regulates traffic coming into a network.
- It sits in front of servers, intercepts client requests and forwards them to backend servers based on predefined rules.
- You can think of a reverse proxy as a **gatekeeper**. Instead of hiding clients from the server, it hides servers from clients.
- Allowing direct access to servers can pose **security risks**, exposing them to threats like **hackers** and **DDoS attacks**. A reverse proxy mitigates these risks by creating a single, controlled point of entry that filters and regulates incoming traffic all while keeping server IP addresses hidden.
- With a reverse proxy in place, clients no longer interact directly with the servers. They only communicate with the reverse proxy.

How a reverse proxy handles a request?

1. A user types a website URL into their browser, which sends a request to the server.
2. The reverse proxy server receives the request before it reaches the backend servers.
3. Based on predefined rules (like load balancing or server availability), the reverse proxy forwards the request to the appropriate backend server.
4. The backend server processes the request and sends a response back to the reverse proxy.

5. The reverse proxy relays the response to the client, with the client never directly interacting with the backend servers.

Key Benefits of Reverse Proxy

1. **Enhanced Security:** By acting as a protective layer, a reverse proxy hides backend servers from clients, reducing the risk of attacks directly targeting backend infrastructure.
2. **Load Balancing:** A reverse proxy can distribute incoming requests evenly across multiple backend servers, improving system reliability and preventing server overload.
3. **Caching Static Content:** Reverse proxies can cache static assets like images, CSS, and JavaScript, reducing the need to fetch these files from the backend repeatedly.
4. **SSL Termination:** Reverse proxies can handle SSL encryption, offloading this work from backend servers.
5. **Web Application Firewall (WAF):** Reverse proxies can inspect incoming requests, acting as a firewall to detect and block malicious traffic.

Setting up a reverse proxy with Nginx:

Nginx is one of the most popular reverse proxy tools. Here's how you can setup a basic reverse proxy configuration using Nginx on a linux server

1. Installing Nginx

```
sudo apt update  
sudo apt install nginx
```

2. **Add Reverse Proxy Configuration**

```
server {  
    listen 80;  
  
    location / {  
        proxy_pass http://backend_server_ip;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_
```

```
x_forwarded_for;
    }
}
```

3. Test and Reload Nginx

`sudo nginx -t sudo systemctl reload nginx`

Load Balancing Across Multiple Servers

It can also implement Load Balancing using different algos like Round Robin, Least Connections, IP Hash ensuring optimal distribution of traffic. By default it uses Round Robin, but we can change that by simply adding the required algo in the upstream block.

With this configuration, Nginx will balance requests among `backend1`, `backend2`, and `backend3`, ensuring no single server becomes overwhelmed.

```
upstream backend_servers {
    ip_hash;
    server backend1.example.com;
    server backend2.example.com;
    server backend3.example.com;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_f
orwarded_for;
    }
}
```



Feature	Proxy Server	Reverse Proxy
Position	Between client and server	Between client and multiple backend servers
Goal	Protect client identity	Distribute requests, load balancing
Use Case	Privacy, bypass restrictions	Load balancing, server protection
Common Users	Individuals, security-focused users	Enterprises, websites with high traffic
Examples	VPN services, browser proxies	Nginx, HAProxy, Cloudflare

▼ 5. HTTP vs HTTPS

Evertime we're interating with the internet, we are using HTTP or its secure counterpart (HTTPS). These protocols are the fundamental languages spoken between clients and servers defining how the information is requested and delivered.

HTTP

1. **Stateless, text-based application layer** protocol
2. used for communication between clients and servers.
3. Client Server Model: Communication is always initiated by the client (for ex: your mobile / web browser sending a request to a server. The server processes the request and sends a response back to the client)
4. **Built on top of TCP:** HTTP traditionally sits on top of TCP leveraging TCPs reliable, ordered and connection oriented delivery.
5. HTTP, by default uses port 80.
6. At its core, HTTP operates on request / response model
 - a. The client first establishes a TCP connection with the server (or reuses an existing one)

- b. Then the client sends HTTP request. It consists of
 - i. HTTP headers (key-value pairs providing additional context)
 - ii. URL
 - iii. HTTP method / verb (GET, PUT, POST, DELETE, PATCH)
 - iv. Message body (optional) for POST requests.
 - c. The server receives the request, processes it (e.g., fetches data from a database, runs business logic).
 - d. The servers then sends the HTTP response back to the client. The response includes:
 - i. HTTP status codes (eg: 200 OK, 404 NOT FOUND etc)
 - ii. Response Headers (eg: Content-Type, Content-Length)
 - iii. Message body (eg: json data)
 - e. Connection Termination: Once the response is received, TCP connection is closed or kept open for subsequent requests (eg: HTTP Keep-Alive)
7. HTTP is inherently stateless which means each request from the client to the server is treated as an independent transaction. The server doesn't remember any previous requests from that client. This is both a blessing and a curse.
- a. Pros: Easier to scale, as any server can handle any request without needing prior context.
 - b. Cons: Requires mechanisms like cookies, session IDs, or JWTs to maintain the user state at the application level.
8. HTTP Methods & Status Codes
- a. Method indicate the desired action to be performed on a resource
 - b. Status Codes indicate the outcome of an HTTP request.
 - i. 1xx (informational): request received or under processing
 - ii. 2xx (success)

- **200 OK** : Request succeeded.
 - **201 Created** : Resource successfully created.
- iii. 3xx (Redirection) — further action is needed to complete the request
- **301 Moved Permanently** : Resource moved.
 - **302 Found** : Temporary redirect.
- iv. 4xx (Client Error)
- **400 Bad Request** : Invalid syntax.
 - **401 Unauthorized** : Authentication required.
 - **403 Forbidden** : Server understood, but refuses to authorize.
 - **404 Not Found** : Resource not found.
- v. 5xx (Server Error)
- **500 Internal Server Error** : Generic server error.
 - **502 Bad Gateway** : Server acting as gateway received invalid response.
 - **503 Service Unavailable** : Server temporarily unable to handle request.

Method	Use Case	Idempotent?
GET	Retrieve data	Yes
POST	Submit new data, create resources	No
PUT	Update/replace an existing resource	Yes
DELETE	Remove a resource	Yes
PATCH	Apply partial modifications to a resource	No
HEAD	Get headers only (no body)	Yes

9. Idempotency is crucial in system design. An operation is called idempotent if it can be performed multiple times without changing the result beyond the initial application. This is vital for retries in distributed systems where

network failures are pretty common. GET, PUT, DELETE are idempotent, while POST and PATCH are generally not.

10. HTTP Limitations

- a. While revolutionary, it had significant limitations that became critical as internet evolved into a platform for sensitive transactions and personal data.
- b. **No encryption:** All data (requests, responses, headers, body) is sent in **plain text**. This means anyone with access to the network path (e.g., your ISP, a public Wi-Fi operator, or a malicious actor) can easily read your information.
- c. **No Authentication:** There's no inherent way to verify the identity of the server you're communicating with. You could be talking to an imposter without knowing.
- d. **No Integrity:** Data transmitted over HTTP can be modified in transit without detection. A malicious actor could change the contents of a webpage or an API response.
- e. These vulnerabilities exposed users to serious security risks like for ex Eavesdropping (confidential info like passwords, credit card numbers) could be stolen. MITM (man in the middle) attacks — an attacker could intercept communication, impersonate both parties and read / alter data. Injection attack like malicious code could be injected.

As web applications scaled and began handling more sensitive data, the need for robust security and privacy became paramount, leading to the development and widespread adoption of HTTPS.

HTTPS

1. HTTP Secure — This is not a separate protocol, but rather HTTP layered on top of TLS / SSL.
 - a. TLS — Transport Layer Security
 - b. SSL — Secure Socket Layer

2. Uses port 443 by default.
3. This cryptographic protocol provides secure communication
4. Provides three core guarantees:
 - a. **Encryption:** All data exchanged between the client and server is encrypted, making it unreadable to third parties who might intercept the traffic. It's like sealing your message in an unbreakable digital envelope.
 - b. **Integrity:** It ensures that the data has not been tampered with or corrupted during transit. If any changes occur, both the client and server will detect them.
 - c. **Authentication:** It verifies the identity of the server (and optionally the client) using digital certificates. This assures the client that they are talking to the legitimate server and not an imposter.
5. Foundation of HTTPS is PKI (public key infrastructure) which uses a combination of Public and Private keys along with the digital certificates to secure communication.
6. HTTPS working:
 - a. secure connection is established using a TLS handshake.
 - b. Client initiates the connection sending a "Client Hello" message that includes its supported TLS version, cipher suites and a random number
 - c. The server responds with a "Server Hello" selecting the best TLS version & cipher suite, along with its own random number and its SSL/TLS certificate.
 - d. The client verifies the server's certificate by checking its validity, expiry date and whether it's issued by a trusted CA (certificate authority). This step ensures that the client is talking to a legitimate server.
 - e. key exchange: the client and server use a public key cryptography to securely exchange a pre-master secret. Both parties then

independently derive a shared symmetric session key using this secret and random numbers.

- f. Once the session key is established, all the subsequent application data (HTTP requests and responses) is encrypted and decrypted using this symmetric session key.

NOTE: Modern TLS (especially TLS 1.3) versions have significantly optimized this handshake process, often allowing zero RTT for clients that have previously connected to a server, reducing latency.

RTT → round trip time.

HTTP vs. HTTPS: Key Differences

Feature	HTTP	HTTPS
Security	Unencrypted, data in plain text	Encrypted via SSL/TLS
Port	80	443
Performance	Slightly faster due to less overhead	Slightly higher latency (improving with TLS 1.3)
Authentication	None	Server authentication via digital certificates
Data Integrity	No guarantee, susceptible to tampering	Guaranteed against tampering
SEO & Trust	Lower ranking, browser warnings, "Not Secure"	Preferred by search engines, padlock icon, user trust
Use Case	Local development, internal, non-sensitive apps	All production web traffic, APIs, sensitive data

NOTE: in modern applications, HTTPS is the undisputed default for all communications, especially in distributed systems handling any form of sensitive data.

Evolution of HTTP Protocols

HTTP has undergone significant evolution to meet the growing demand of the web.

HTTP / 1.1

- Introduced **persistent connections** (keep-alive), allowing multiple requests over a single TCP connection, reducing overhead.
- Enabled **caching mechanisms** and **virtual hosting**.
- **Limitation:** Still suffered from **head-of-line blocking**. Even with persistent connections, requests were processed sequentially. If one request was slow, it blocked all subsequent requests on that connection.

HTTP/2 (2015)

- Addressed HTTP/1.1's limitations by introducing binary framing, multiplexing, and header compression.
- Binary Framing: Data is no longer sent as plain text but in a binary format, making it more efficient to parse.
- Multiplexing: Allows multiple concurrent requests and responses over a *single TCP connection*, eliminating head-of-line blocking.
- Header Compression (HPACK): Reduces redundant header data, saving bandwidth.
- Server Push: Servers can proactively send resources to the client that it anticipates the client will need (e.g., CSS before it's requested in the HTML).
- System Design Impact: Significantly improved latency and efficiency for loading complex web pages with many assets. This is the foundation for many modern APIs and microservices.

HTTP/3 (2018, RFC 9114)

- The most radical change: it runs over UDP (User Datagram Protocol) instead of TCP, utilizing the QUIC (Quick UDP Internet Connections) protocol.
- Why UDP?: To overcome TCP's own head-of-line blocking (where a single lost packet on a TCP connection can stall all streams on that connection) and reduce handshake latency.

- 0-RTT Connection Establishment: For repeat connections, HTTP/3 can often send data in the very first packet, dramatically reducing setup latency.
- Connection Migration: A client can seamlessly switch networks (e.g., from Wi-Fi to cellular) without dropping the connection, as QUIC connections are identified by a unique ID, not IP/port.
- System Design Impact: Ideal for real-time applications, faster connection establishment, and improved performance on unreliable networks. Google, YouTube, and Cloudflare are major adopters.

▼ 6. TCP vs UDP

When you send a WhatsApp message, stream a YouTube video, or play an online game, your device relies on a set of rules to move data across the internet.

At the heart of this process are two fundamental protocols: **TCP** and **UDP**. They are the invisible couriers that decide *how* your data is packaged and sent.

They both operate at Layer 4 of the OSI model. Their job is to take data from an application on one device and deliver it to an application on another.

Choosing between them is a critical design choice based on following scenarios:

1. Do we need a courier that guarantees every single piece of your message arrives in perfect order, even if it takes a bit longer.
2. Do we prefer the one which delivers as fast as possible, even if a few pieces gets lost along the way.

This choice is a constant balancing act between **speed**, **reliability**, and **resource usage**.

Let's first briefly understand what transport layer does?

While the network layer(IP) gets the packets from one computer to another computer (host to host), the transport layer gets data from one application to another (process to process)

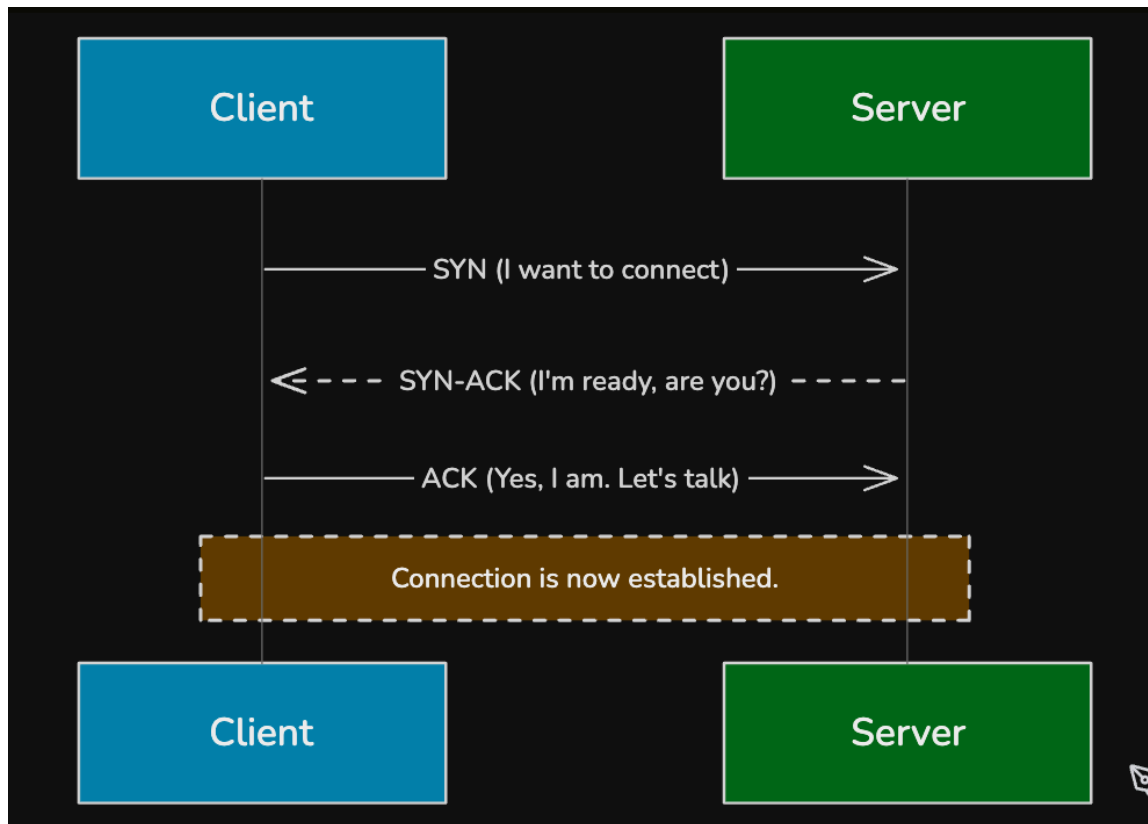
Key responsibilities include:

- **Segmentation and Reassembly:** Breaking large chunks of application data into smaller packets on the sender's side and reassembling them on the receiver's side.
- **Delivery Model:** Providing either a **reliable delivery** service or a **best-effort** transmission.
- **Flow and Congestion Control:** Managing the rate of data transmission to avoid overwhelming the receiver or the network itself.

This is where TCP and UDP diverge. **TCP focuses on providing a highly reliable, ordered stream of data, while UDP prioritizes speed and simplicity with a best-effort approach.**

TCP

1. It's a connection oriented protocol
2. guarantees the reliable delivery of data in the correct order.
3. Before sending any data, TCP establishes a formal connection between the client and server. TCP uses a **three-way handshake** (SYN, SYN-ACK, ACK) to establish a reliable connection. This initial negotiation adds a bit of latency but ensures both sides are ready to communicate.
4. **Reliability:** Every packet sent is tracked with a sequence number.
5. The receiver sends acknowledgments (ACKs) for packets it receives. If the sender doesn't get an ACK within a certain time, it **retransmits** the lost packet.
6. **Ordered delivery:** Sequence numbers also ensure that packets are reassembled in the correct order at the destination, even if they arrive out of order.
7. **Flow & Congestion control:** TCP uses a "sliding window" mechanism to prevent the sender from overwhelming the receiver (**flow control**).
8. It also intelligently slows down transmission when it detects network congestion (**congestion control**).



Connection Establishment (Three way handshake)

- **SYN (Synchronize):** The client initiates the connection by sending a SYN message, signaling its intent to start communication and sharing its initial sequence number (ISN).
- **SYN-ACK (Synchronize–Acknowledge):** The server receives the SYN, reserves resources for the connection, and responds with a SYN-ACK to acknowledge the request and share its own ISN.
- **ACK (Acknowledge):** The client sends an ACK to confirm receipt of the SYN-ACK, completing the handshake.
- At this point, both sides have synchronized sequence numbers, and a reliable communication channel is established.

Data Transmission

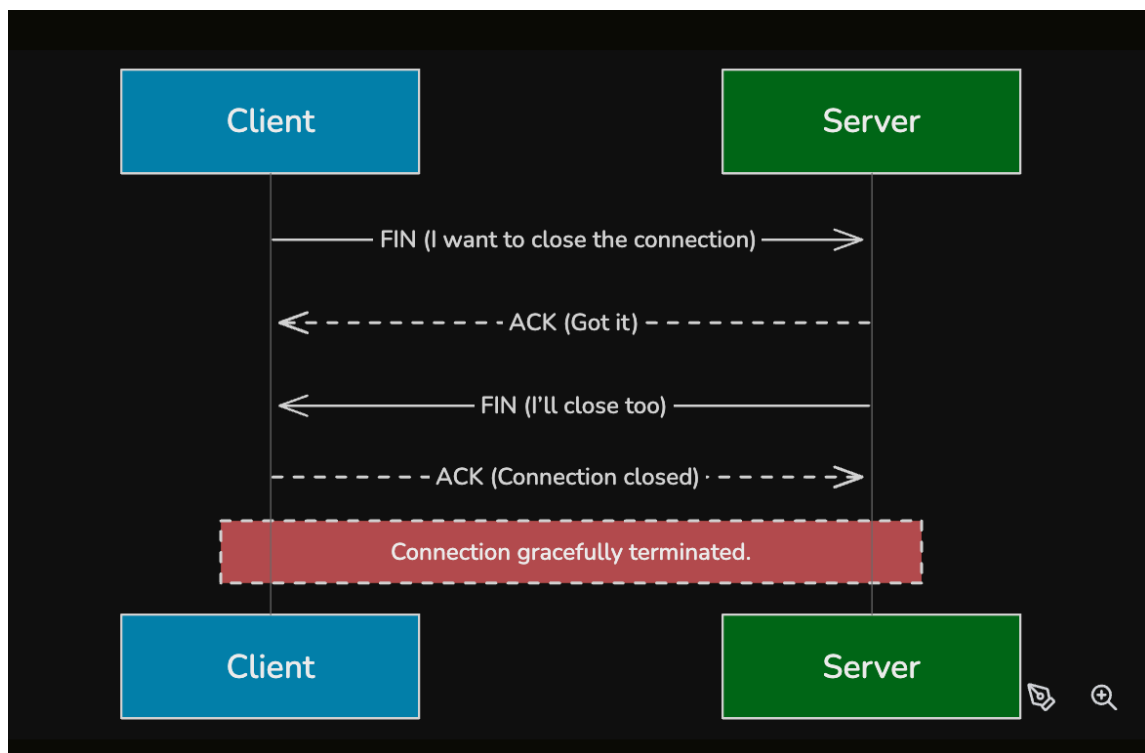
- Once the connection is established, TCP begins transmitting data. The protocol guarantees that all packets (segments) are delivered **in order, without duplication, and without loss.**

- **Segmentation:** Large data is broken into smaller, manageable chunks called *segments*.
- **Acknowledgments (ACKs):** Each segment sent is acknowledged by the receiver. If the sender doesn't receive an ACK within a set timeout, it retransmits the segment.
- **Sliding Window:** TCP uses a window-based flow control mechanism that allows multiple packets to be "in flight" before requiring an acknowledgment, improving throughput.
- **Error Detection:** Each segment includes a checksum to detect corruption in transit. Corrupted packets are discarded and resent.

In essence, TCP behaves like a reliable courier service: it delivers every packet, ensures they arrive in the right order, and resends any that get lost.

Connection Termination

- When communication is complete, TCP closes the connection gracefully through a four-step termination process. This ensures both client and server agree that the session is finished.



- **FIN (Finish):** The client initiates termination, signaling it has no more data to send.
- **ACK:** The server acknowledges the FIN, allowing any remaining data to be processed.
- **FIN:** The server sends its own FIN to close its side of the connection.
- **ACK:** The client confirms, and both sides release their resources.

This **graceful shutdown** ensures that no data is lost and both parties are aware the connection is closed.

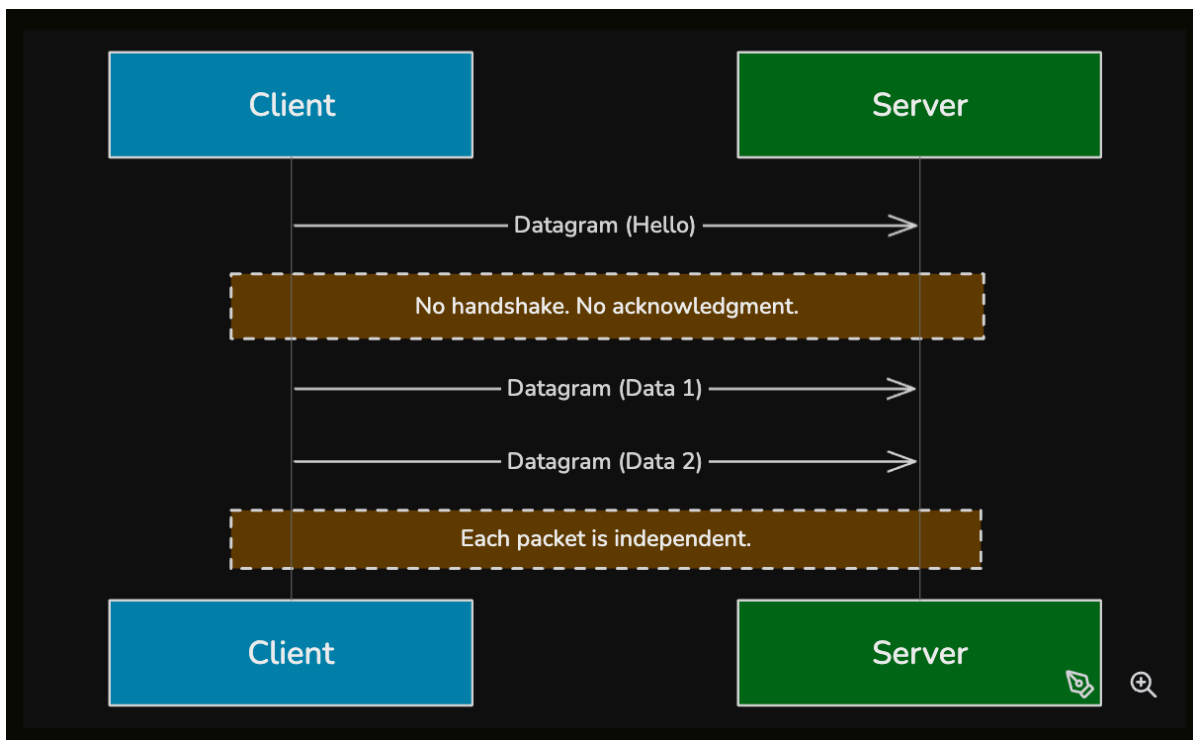
UDP

1. UDP is a **connectionless** protocol that sends data without establishing a formal connection.
2. It operates on a "fire-and-forget" principle.
3. it's the fast & no-frills courier of the internet.
4. No handshake: Packets or datagrams are sent immediately without any prior negotiation. This significantly reduces initial latency.
5. No acknowledgements: it doesn't wait for ACKs and doesn't retransmit lost packets. If a packet is dropped, it's gone for good.
6. There's no guarantee that packets will arrive in the order they were sent.
7. The UDP header is much smaller (8 bytes) than the TCP header (20+ bytes), meaning less overhead per packet.

UDP working:

- UDP doesn't perform a handshake like TCP. So there's no setup phase, no exchange of sequence numbers, and no acknowledgment of connection readiness.
- When an application wants to send data, It creates a **datagram**, attaches a destination IP and port, and sends it directly over the network. The receiving application simply processes the incoming data.

- This simplicity eliminates the connection overhead, allowing UDP to deliver data almost instantaneously.
- UDP treats every packet (called a **datagram**) as an independent message.
- Each datagram carries its own header information and is routed individually through the network. This makes UDP ideal for real-time communication where dropping a few packets is acceptable but delays are not.



Head to head comparison:

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Type	Connection-oriented	Connectionless
Reliability	Reliable (acknowledged delivery)	Unreliable (no ACKs, "best-effort")
Order	Guaranteed (sequenced packets)	Not guaranteed

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Speed	Slower due to handshakes and overhead	Faster, minimal overhead
Packet Loss Handling	Retransmits lost packets	Ignores lost packets
Flow & Congestion Control	Yes	No
Use Cases	Web, Email, File Transfer, APIs, Databases	Streaming, Gaming, VoIP, DNS

In essence, TCP prioritizes correctness while UDP prioritizes timeliness.

Modern Innovations:

The classic TCP vs. UDP debate has evolved. Modern protocols aim to get the best of both worlds.

QUIC (Quick UDP Internet Connections)

1. developed by Google and forms the basis of HTTP/3
2. QUIC is the gamechanger, why because it runs over UDP, so it avoids the initial latency of the TCP handshake. At the same time it provides reliability, congestion control and stream management directly within its own layer.
3. Features built in mandatory encryption(TLS 1.3)
4. It supports multiplexing as well, where multiple data streams can be sent over a single connection without one blocking the others.

Essentially, **QUIC provides many of TCP's benefits with the speed of UDP.**

Some applications also build their own **custom reliability layers** over UDP. For example, a multiplayer game might use UDP for player movement but implement a simple ACK system for critical events like "player used a special ability."

Choosing the Right Protocol

There are multiple factors that need to be considered while choosing between TCP & UDP.

1. Latency

- a. **TCP**: Introduces latency upfront with its three-way handshake. Every acknowledgment and potential retransmission adds to the round-trip time (RTT).
- b. **UDP**: Has near-zero initial latency. Data is sent immediately. This is a huge advantage for time-sensitive applications.

2. Throughput:

- a. TCP's congestion control mechanism can throttle throughput to adapt to network conditions. This is good for the overall health of the system but it can limit the maximum speed of a connection.
- b. On the other hand, UDP can achieve higher throughput as it doesn't slowdown for congestion control. The risk is that sending too fast can lead to more packet loss.

3. Reliability vs Efficiency

- a. TCP provides reliability for free at the transport layer but uses more CPU, memory, and bandwidth to manage connections and state.
- b. UDP is highly efficient & extremely fast but pushes the responsibility of handling packet loss and ordering up to the application layer.

4. Security:

- a. TCP is the foundation of TLS(transport layer security) which powers HTTPS. The reliable, ordered nature of TCP is a prerequisite for how TLS works.
- b. UDP does not have a built-in mechanism for security. Applications must use a protocol like **DTLS (Datagram TLS)** to add a layer of encryption.

NOTE: Many large-scale systems use a **hybrid approach**: They use TCP for authentication and API calls. They use UDP for sending non-essential

telemetry and metrics and QUIC / HTTP/3 for real time streaming and user facing web content.

Decision Framework

If your application requires...	Then use...
Guaranteed & Ordered Delivery	TCP
The Lowest Possible Latency	UDP (or QUIC)
Tolerance for some Packet Loss	UDP
High Accuracy and Data Integrity	TCP
Real-Time Communication	UDP
Built-in Flow/Congestion Control	TCP

Real world usecases:

TCP

1. **Web traffic (HTTP / HTTPS):** When you load a web page, every single byte of HTML, CSS, and JavaScript must arrive correctly and in order. A missing packet could break the page layout or functionality.
2. **Databases:** A database query or transaction must be 100% complete and correct. We can't afford to have a partially updated record due to a lost packet.
3. **Email (SMTP, IMAP):** Just like a web page, an email message must be delivered in its entirety to be readable.

UDP:

1. **Video streaming (youtube / netflix):** if a single frame of a video is dropped, user might see a glitch or nothing at all. It's better to skip that frame and show the next one in realtime than to pause the entire frame to wait for a retransmission.

2. **Online Gaming:** In a fast-paced game, receiving slightly outdated information is better than receiving perfect information too late. The game client can often interpolate or predict the missing data.
3. **Voice over IP (VoIP - Zoom, WhatsApp Calls):** A small, momentary audio dropout is preferable over a long, jarring delay while the system tries to recover a lost packet.

▼ 7. Checksums

lets first understand the concept of checksums. Imagine that you're sending an imp letter to your friend. Before sealing the envelope, you took a photo of the letter. When your friend receives the letter, they also take its photo and send it back to you. If the two photos match, you can confirm that the letter hasn't been tampered / altered. But if they don't match, its a clear sign that something went wrong along the way.

In the digital world, checksums serve a similar purpose as those photos.

Just like taking photos help us answer the question: **"Has the letter been altered or damaged"**, a checksum answers the question: **"Has this data been altered unintentionally or maliciously since it was created, stored, or transmitted?"**

1. Checksum is a unique fingerprint attached to the data before it's transmitted.
2. When the data arrives at the recipient's end, the fingerprint is recomputed to ensure it matches the original one.
3. If the checksum matches, you can be confident that the data hasn't been modified or damaged.
4. Checksums are calculated by performing a mathematical operation on the data, such as adding all the bytes or running it thru the cryptographic hash function.

How does Checksum work?

1. **Checksum Calculation:** before storing or sending data, the original data is processed thru a specific algo to produce a checksum value.

2. The checksum is appended to the data and sent over the network or saved in storage.
3. **Verification:** Upon retrieval or reception, the checksum is recalculated using the same algorithm on the received data. This newly calculated checksum is compared with the original checksum.
4. **Error Detection:** If the two checksum values match, the data is considered intact. If they do not match, it indicates that the data has been altered or corrupted during transmission or storage.

Types of CheckSums:

there are several types of checksums each with its own strengths and weaknesses.

1. Parity Bit: A parity bit is a single bit that is added to a group of bits to make the total number of 1s either even (even parity) or odd (odd parity). While it can detect single bit errors, it fails if an even number of bits are flipped.
2. CRC (cyclic redundancy checks): the way it works is by treating the data as a large binary number and dividing it by a predetermined divisor. Remainder becomes the checksum
3. Cryptographic Hash Functions: These are one-way functions that generate a fixed-size hash value from the data. For ex: SHA-1, SHA-256, MD5

In essence, checksums serve as a vital line of defense, safeguarding against errors and corruption. Seamlessly performs error detection and provides confidence to the clients on the data integrity and accuracy.

▼ 8. Load Balancing

Load Balancer is a system that spreads incoming network traffic across multiple backend servers (often called "worker nodes" or "application servers").

It ensures that no single server becomes a bottleneck due to an overload of requests.

By distributing the load, applications can handle higher volumes of traffic and remain robust in the face of server failures.

Why Do We Need a Load Balancer?

1. **Scalability:** As traffic grows, we can add more servers behind the load balancer without redesigning our entire architecture.
2. **High Availability:** If one server goes offline or crashes, the load balancer automatically reroutes traffic to other healthy servers.
3. **Performance Optimization:** Balancing load prevents certain servers from overworking while others remain underutilized.

Types of Load Balancers

Load balancers can be categorized in a few ways:

Hardware Load Balancers: These are specialized physical devices often used in data centers. for ex: Citrix ADC, F5 etc. They tend to be very powerful, but can be expensive and less flexible.

Software Load Balancers: These run on standard servers or virtual machines. For ex: Nginx, HAProxy, Envoy etc. These are often open-source or lower-cost solutions, highly configurable, and simpler to integrate with cloud providers.

Layer 4 Load Balancers: These lb's distribute the traffic based on network information like IP addresses or ports. It doesn't inspect the application layer data like HTTP, HTTP headers etc.

Layer 7 Load Balancers: These lb's make decisions based on application layer data like HTTP headers, cookies, URLs etc. This is useful for advanced routing and application aware features.

How Load Balancing Works

1. **Traffic Reception:** All incoming requests arrive at the load balancer's public IP or domain
2. **Routing Algorithms:** The load balancer decides which server should get the request. Common routing algorithms include:
 - a. **Round Robin:** Requests are distributed sequentially to each server in a loop.

- b. **Weighted Round Robin:** Each server is assigned a weight (priority). Servers with higher weights receive proportionally more requests.
- c. **Least Connections:** The request goes to the server with the fewest active connections.
- d. **IP Hash:** The load balancer uses a hash of the client's IP to always route them to the same server (useful for sticky sessions).
- e. **Random:** Select a server randomly

3. **Server Health Checks:** Load balancers usually have an internal mechanism to periodically check if servers are alive (e.g., by sending a heartbeat request like an HTTP `GET /health`).

- If a server doesn't respond within a certain threshold, it's marked as **unhealthy** and no longer receives traffic.
- When it recovers, the load balancer can automatically reintroduce it into the rotation.

4. **Response Handling:** Once a request is forwarded to a healthy server, the server processes it and returns the response to the load balancer, which then relays it to the client.



Some load balancers forward responses directly, while others might modify headers or terminate SSL (HTTPS) connections for performance reasons.

Key Features

1. **SSL / TLS termination:** The lb offloads cryptographic operations from the backend servers. It decrypts the incoming SSL traffic and sends plain HTTP traffic to backend servers, reducing their CPU load.
2. **Sticky Sessions / Session Persistence:** This ensures that a client's request are always routed to the same server. This can be important for Stateful applications (though it's often better to design stateless services and store state externally , eg: in a cache)

3. Auto Scaling: In cloud env, we can integrate the lb's with autoscaling groups. As traffic increases, new servers spin up and they automatically register themselves with the lb.
4. Caching and Compression: Some lb's can cache responses (especially application level ones) and also compress them to reduce bandwidth.
5. Enhanced Security: Modern lb's often support security features like WAF (web application firewall), DDOS protection etc.

Real world usecases:

1. API Gateways: Modern microservices often place a load balancer or reverse proxy in front of internal services to route API calls based on route paths or hostnames.
2. Ecommerce: During peak sale events like Black Friday, a sudden traffic spikes occur. A load balancer helps distribute that traffic evenly and ensures the system can scale horizontally by adding more servers.

Conclusion:

A lb is often at the heart of any modern distributed system. It ensures that no single server is overwhelmed with requests and that our application remains available — even when individual servers fails.

By using load balancers effectively, you can:

- **Scale out** your infrastructure seamlessly.
- **Enhance availability** and fault tolerance.
- **Improve performance** by offloading tasks like SSL termination and caching.
- **Maintain** your systems more easily, taking servers offline without impacting the entire application.

▼ **8.1 Load Balancing Algorithms**

- ▼ Round Robin:

- When all servers have similar processing capabilities and are equally capable of handling requests.
- When simplicity and even distribution of load is more critical.

Pros:

- Easy to implement
- Ensures even traffic distribution

Cons:

- Doesn't take into account server load or response times.
- Can lead to inefficiencies if servers have different processing capabilities.

Implementation:

```
class RoundRobin:
    def __init__(self, servers):
        self.servers = servers
        self.current_index = -1

    def get_next_server(self):
        self.current_index = (self.current_index +
1) % len(self.servers)
        return self.servers[self.current_index]

# Example usage
servers = ["Server1", "Server2", "Server3"]
load_balancer = RoundRobin(servers)

for i in range(6):
    server = load_balancer.get_next_server()
    print(f"Request {i + 1} -> {server}")
```

▼ Weighted Round Robin

When servers have different processing capabilities or available resources.

Pros:

- Balances load according to server capacity.
- More efficient use of server resources.

Cons:

- Doesn't take into account server load or response times.

Implementation:

```
from math import gcd
from functools import reduce

class WeightedRoundRobin:
    def __init__(self, servers, weights):
        self.servers = servers
        self.weights = weights
        self.current_index = -1
        self.current_weight = 0

    def _gcd_of_weights(self):
        return reduce(gcd, self.weights)

    def get_next_server(self):
        gcd_val = self._gcd_of_weights()
        while True:
            self.current_index = (self.current_index +
1) % len(self.servers)
            if self.current_index == 0:
                self.current_weight -= gcd_val
                if self.current_weight <= 0:
                    self.current_weight = max(self.weights)
            if self.weights[self.current_index] >= sel
```

```

f.current_weight:
    return self.servers[self.current_index]

# Example usage
servers = ["Server1", "Server2", "Server3"]
weights = [5, 1, 1]
lb = WeightedRoundRobin(servers, weights)

for i in range(1, 8):
    print(f"Request {i} -> {lb.get_next_server()}")

```

▼ Least Connections

- When you want to distribute the load based on the current number of active connections.
- The algorithm tracks the no of active connections on each server. It assigns the request to the server with least active connections

Pros:

- Balances load more dynamically based on current server load

Cons:

- Requires tracking active connections for each server

Implementation:

```

import random

class LeastConnections:
    def __init__(self, servers):
        self.connections = {server: 0 for server in servers}

    def get_next_server(self):
        min_conn = min(self.connections.values())

```

```

        least_loaded = [s for s, c in self.connections.items() if c == min_conn]
        selected = random.choice(least_loaded)
        self.connections[selected] += 1
        return selected

    def release_connection(self, server):
        if self.connections[server] > 0:
            self.connections[server] -= 1

# Example usage
servers = ["Server1", "Server2", "Server3"]
lb = LeastConnections(servers)

for i in range(1, 7):
    server = lb.get_next_server()
    print(f"Request {i} -> {server}")
    lb.release_connection(server)

```

when creating a new connection: increments the connection count for that server

when a connection is closed: decrementing the connection count for the corresponding server.

▼ Least Response Time

- when we have servers with varying response times and want to route requests to the fastest server.
- The algo monitors the response time of each server. It assigns requests to the server with the fastest response time.

Pros:

- Minimizes overall latency by selecting the server with the fastest response time. Helps improve the user experience by providing quick responses.

- Can adapt dynamically to changes in server response times.

Cons:

- Requires accurate measurement of server response times, which can be challenging in distributed systems.
- May not consider other factors such as server load or connection count.

Implementation:

```
import random

class LeastResponseTime:
    def __init__(self, servers):
        self.servers = servers
        self.response_times = [0.0] * len(servers)

    def get_next_server(self):
        min_index = self.response_times.index(min(self.response_times))
        return self.servers[min_index]

    def update_response_time(self, server, response_time):
        index = self.servers.index(server)
        # Exponential weighted moving average (alpha = 0.3)
        self.response_times[index] = 0.7 * self.response_times[index] + 0.3 * response_time

# Example usage
servers = ["Server1", "Server2", "Server3"]
lb = LeastResponseTime(servers)

random.seed(42)
for i in range(1, 7):
```

```

server = lb.get_next_server()
response_time = random.uniform(10, 200)
lb.update_response_time(server, response_time)
print(f"Request {i} -> {server} ({response_time:.0f}ms)")

```

`simulate_response_time()` function that introduces a random delay to mimic the server's response time.

`get_next_server()` method selects the server with the least response time. The `update_response_time()` method is called after each request to update the response time for the corresponding server.

▼ IP Hash

- When we need session persistence, as requests from the same client are always directed to the same server.
- It calculates the hash of the client's IP address and uses that to determine the server to route the request.

Pros:

- useful for applications that require sticky sessions

Cons:

- can lead to uneven load distribution if certain IP addresses generate more traffic than others.
- Less flexibility if the server goes down. Hash mapping may need to be reconfigured.

Implementation:

```

import hashlib

class IPHash:
    def __init__(self, servers):
        self.servers = servers

    def get_next_server(self, client_ip):

```

```

        hash_value = hashlib.md5(client_ip.encode()).hexdigest()
        index = int(hash_value, 16) % len(self.servers)
        return self.servers[index]

# Example usage
servers = ["Server1", "Server2", "Server3"]
lb = IPHash(servers)

client_ips = ["192.168.0.1", "192.168.0.2", "192.168.0.3", "192.168.0.4"]
for ip in client_ips:
    print(f"Client {ip} -> {lb.get_next_server(ip)}")

# Demonstrate consistency: same IP always maps to same server
print("\nConsistency check:")
for _ in range(3):
    print(f"192.168.0.1 -> {lb.get_next_server('192.168.0.1')}")

```

This also ensures that **requests from the same IP address are always directed to the same server** useful for stateful applications